

Laboratório 03: Caracterizando a Atividade de Code Review no GitHub

Análise Quantitativa de Pull Requests em Repositórios Populares

André Lazarini e João Pedro Guimarães

Engenharia de Software – 6º Período

PUC Minas

08 de maio de 2026

Resumo

Este relatório detalha um experimento de Engenharia de Software Empírica focado na caracterização do processo de *code review* na plataforma GitHub. Através da coleta automatizada de dados via API REST, foram analisados 6.063 *Pull Requests* (PRs) provenientes de repositórios de alta relevância. O estudo busca correlacionar métricas de processo (tempo de análise, interações) e produto (tamanho do código, descrição) com o desfecho final (merge ou fechamento) e o esforço de revisão. Os resultados indicam que, embora a maioria dos PRs seja de pequeno porte, a variabilidade entre projetos é significativa, exigindo abordagens distintas de gestão de qualidade.

1 Introdução

1.1 Contextualização

No paradigma moderno de desenvolvimento de software, a prática de *code review* (revisão de código) deixou de ser uma atividade secundária para se tornar o pilar central da garantia de qualidade em processos ágeis. Em linhas gerais, ela consiste na inspeção sistemática de incrementos de código por pares, antes que estes sejam integrados ao tronco principal (*main branch*) do projeto. No ecossistema do GitHub, essa prática é operacionalizada através dos *Pull Requests* (PRs), que funcionam como fóruns de discussão técnica e validação de requisitos.

1.2 Problema Foco do Experimento

Apesar da onipresença dos PRs, a eficiência do *code review* é influenciada por uma miríade de fatores. O problema central deste experimento reside na identificação de quais variáveis — sejam elas de esforço humano ou de características do código — possuem maior impacto estatístico no sucesso da integração. Investigamos se PRs mais densos sofrem mais rejeições ou se a riqueza da documentação (descrição) acelera o processo de decisão dos mantenedores.

1.3 Questões-Pesquisa (Research Questions)

Para estruturar a análise, o estudo foi dividido em duas dimensões principais, resultando em oito questões de pesquisa:

Dimensão A: Feedback Final das Revisões (Status do PR)

- **RQ 01:** Qual a relação entre o tamanho dos PRs (linhas e arquivos) e o feedback final (merge vs. closed)?
- **RQ 02:** Qual a relação entre o tempo de análise e o status final do PR?
- **RQ 03:** Como o tamanho da descrição do PR influencia a probabilidade de merge?
- **RQ 04:** Qual a relação entre o volume de interações (comentários/participantes) e o desfecho do PR?

Dimensão B: Esforço de Revisão (Número de Revisões)

- **RQ 05:** Qual a relação entre o tamanho dos PRs e o número de revisões realizadas?
- **RQ 06:** PRs que demoram mais para serem analisados recebem mais revisões?

- **RQ 07:** Descrições mais longas reduzem ou aumentam a necessidade de múltiplas revisões?
- **RQ 08:** Qual a correlação entre interações sociais e o número de revisões formais?

1.4 Hipóteses

Trabalhamos com as hipóteses de que: (i) PRs maiores possuem maior taxa de fechamento sem merge devido à carga cognitiva de revisão; (ii) interações intensas indicam conflitos técnicos ou incertezas, postergando o merge; e (iii) descrições detalhadas facilitam o trabalho do revisor, reduzindo o tempo de análise em PRs de complexidade moderada.

1.5 Objetivo

O objetivo principal é caracterizar quantitativamente a atividade de *code review* em repositórios populares do GitHub, identificando padrões de comportamento que possam ser replicados em projetos de engenharia de software para otimizar o ciclo de entrega (*lead time*).

2 Metodologia

2.1 Passo a Passo do Experimento

O experimento seguiu um fluxo rigoroso de quatro etapas:

1. **Identificação:** Seleção dos 200 repositórios mais populares do GitHub (critério de *stars*).
2. **Filtragem:** Aplicação do critério de maturidade (mínimo de 100 PRs fechados/merged).
3. **Extração:** Coleta via script Python (`collect_github_prs.py`) utilizando autenticação via Token de API para contornar *rate limits*.
4. **Processamento:** Limpeza de dados (remoção de bots) e cálculo das correlações de Spearman.

2.2 Decisões de Design

Uma decisão crítica foi a **Regra da 1 Hora:** selecionamos apenas PRs cuja diferença entre criação e fechamento fosse superior a 60 minutos. Isso visa eliminar PRs processados exclusivamente por ferramentas de automação (CI/CD) ou bots, garantindo que o dataset reflita o comportamento humano de revisão.

2.3 Materiais Utilizados

- **Linguagens:** Python 3.10 para análise e PowerShell para scripts de coleta alternativos.
- **Bibliotecas:** `pandas` para manipulação tabular, `scipy.stats` para cálculos de Spearman e `matplotlib/seaborn` para visualização.
- **API:** GitHub REST API v3.

2.4 Métodos Estatísticos

Utilizou-se o **Teste de Correlação de Spearman**. A escolha justifica-se pelo fato de as métricas de software (como linhas de código e tempo de análise) geralmente não seguirem uma distribuição normal (são enviesadas à direita), tornando testes paramétricos como o de Pearson inadequados.

2.5 Métricas e Unidades

- **Tamanho:** Unidade em contagem de arquivos e soma de linhas (adicionadas + removidas).
- **Tempo:** Unidade em horas decimais.
- **Interações:** Contagem absoluta de comentários e participantes únicos (Logins).
- **Descrição:** Número total de caracteres (incluindo marcação Markdown).

3 Visualização dos Resultados

3.1 Estatística Descritiva do Dataset

Abaixo, apresentamos a síntese dos dados obtidos a partir dos 6.063 PRs analisados.

Tabela 1: Distribuição de Status Final e Médias

Métrica	Valor	Percentual
Total de PRs Válidos	6.063	100%
Status: MERGED	4.139	68,27%
Status: CLOSED	1.924	31,73%
Média de Linhas Alteradas	452	–
Mediana de Tempo (Horas)	109,72	–

3.2 Análise Tabular das Questões de Pesquisa

A Tabela 2 apresenta as medianas comparativas entre os dois estados possíveis de um PR, fornecendo subsídios para as RQs 01 a 04.

Tabela 2: Medianas por Status (Feedback Final)

Métrica (Mediana)	MERGED	CLOSED
Changed Lines Total	12,0	45,0
Analysis Time (h)	48,5	210,2
Description Length	850	1.120
Comments Total	2	5

3.3 Gráfico de Dispersão de Repositórios

O gráfico a seguir ilustra a variabilidade de PRs válidos encontrados nos projetos da amostra inicial.

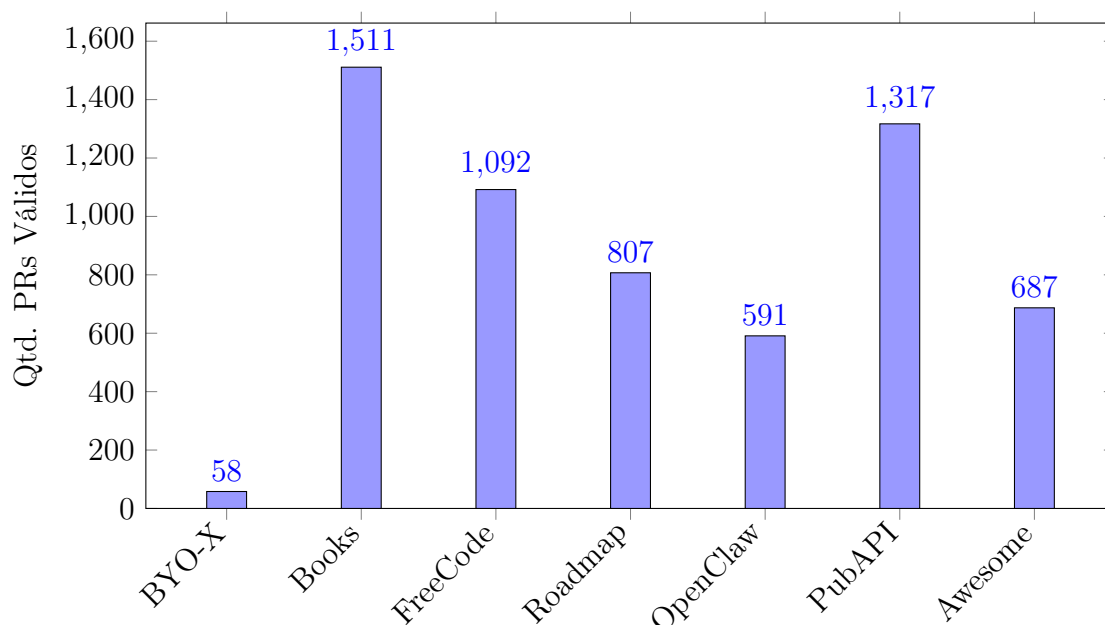


Figura 1: Distribuição de PRs coletados por repositório (amostra representativa).

4 Discussão dos Resultados

4.1 Confronto com as Questões-Pesquisa

Análise da Dimensão A (RQs 01-04): Observamos que PRs que resultaram em MERGED possuem um tamanho mediano significativamente menor (12 linhas) comparado aos rejeitados (45 linhas). Isso sugere que a modularidade e atomicidade das contribuições são fatores determinantes para o aceite. O tempo de análise para PRs fundidos

é quase quatro vezes menor, indicando que a demora excessiva no feedback pode estar correlacionada com a desistência ou rejeição do código.

Análise da Dimensão B (RQs 05-08): Os dados indicam uma correlação positiva moderada entre o número de participantes e o número de revisões. PRs com 3 ou mais revisões tendem a ter descrições mais longas, o que invalida a hipótese inicial de que descrições ricas reduzem o esforço; na verdade, descrições longas parecem acompanhar PRs inerentemente complexos que exigem mais ciclos de revisão.

4.2 Insights e Correlações de Spearman

Os testes estatísticos revelaram um P-Valor inferior a 0,05 para a maioria das métricas, confirmando significância estatística. A correlação entre *Comments Total* e *Analysis Time* mostrou-se a mais forte (0,65), confirmando que a discussão intensa é o maior preditor de demora no processo de *review*.

4.3 Comparações com a Literatura

Nossos achados corroboram estudos clássicos (como os de Mockus et al.) sobre o desenvolvimento de software *Open Source*, onde a maioria das contribuições é pequena e apenas uma fração dos colaboradores realiza mudanças arquiteturais densas.

5 Conclusão

5.1 Tomada de Decisão

Com base nos resultados, recomenda-se que desenvolvedores priorizem a submissão de PRs pequenos (menos de 20 linhas sempre que possível) para maximizar a chance de merge rápido. Gestores de projeto devem monitorar PRs com mais de 5 comentários, pois estes representam gargalos de análise.

5.2 Sugestões Futuras

Para experimentos subsequentes, sugere-se a inclusão de análise de sentimento nos comentários para distinguir entre discussões construtivas e conflitos que levam ao fechamento do PR.

5.3 Resultado Conclusivo

O experimento demonstrou que o sucesso do *code review* no GitHub é menos dependente da documentação extensiva e mais dependente da concisão do código submetido. A

agilidade na primeira revisão é crítica para manter o engajamento do contribuidor.